

TravelLink Backend — Progress Note

Last Updated: 2026-05-31 | **Tech Stack:** .NET 8, EF Core, SQL Server, SignalR, Gemini AI, Cloudinary, Open-Meteo

Project Structure

```
Server/  
├─ Controllers/      AuthController, UserController,  
FriendController,  
|                   GroupController, ExpenseController,  
ItineraryController  
├─ Services/         (Interface + Implementation for each module)  
├─ Models/Entities/  12 entity models  
├─ DTOs/             Organized by module  
├─ Data/             AppDbContext  
├─ Hubs/             LocationHub (SignalR)  
├─ Helpers/  
└─ Common/           ServiceResult<T> wrapper
```

✓ Module 1 — Authentication

Service: AuthService | **Controller:** AuthController

- JWT-based login & registration
 - Password hashing
 - Token generation with `ClaimTypes.NameIdentifier` (Guid userId)
-

✓ Module 2 — User

Controller: UserController

- Get user profile

- Update profile (name, bio, etc.)
 - Profile image upload via Cloudinary (`ImageService`)
-

✓ Module 3 — Friend Management

Service: `FriendService` | **Controller:** `FriendController`

Endpoint	Description
Send friend request	Creates <code>FriendRequest</code> entity
Accept / Reject	Updates request status
Unfriend	Social constraint — only friends can unfriend
List friends	Returns accepted connections

Constraint: Cannot unfriend someone who isn't your friend (guard added).

✓ Module 4 — Group Management

Service: `GroupService` | **Controller:** `GroupController`

Endpoint	Description
Create group	Admin auto-assigned to creator
Update group info	Admin only
Upload group image	Cloudinary
Add/Remove members	Admin only
Leave group	Auto-deletes group if last member leaves
List group members	

✓ Module 5 — Image Storage

Service: ImageService | Uses Cloudinary

- User profile pictures
 - Group profile pictures
 - Upload returns URL stored in DB
-

✓ Module 6 — Expense Module

Service: ExpenseService | **Controller:** ExpenseController

Endpoint	Description
Add expense	Splits among selected members
Delete expense	
Settle up	Mark splits as settled
Get group expenses	
Get balance summary	Per-person net owed/owes
Analytics dashboard	Spend by category, total, per person

Edge cases handled:

- Solo expenses (paid by self, for self)
 - Third-party payments
 - Correct split calculation with rounding
-

✓ Module 7 — Real-Time Location

Hub: LocationHub (SignalR)

- Privacy-first design
 - Members share live GPS coordinates within their group
 - Coordinates broadcast only to group members
 - Angular frontend will consume via SignalR client
-

✓ Module 8 — Smart Itinerary Builder

Service: `ItineraryService` (largest module, ~750 lines)

Controller: `ItineraryController`

External APIs: Gemini 1.5 Flash, Open-Meteo (weather + geocoding)

8A — Entities

Entity	Key Fields
<code>ItineraryRequest</code>	GroupId, Destination, StartDate, EndDate, TotalBudget, VehicleType, VehicleCount, IsRental, DailyRentalCostPerVehicle, DailyFuelCostPerVehicle, DailyHotelCostPerRoom, NumberOfRooms, GroupSize, Status
<code>PlaceSuggestion</code>	ItineraryRequestId, Name, Type, Notes, AdminApproved
<code>SuggestionVote</code>	SuggestionId, UserId
<code>GeneratedItinerary</code>	ItineraryRequestId, TotalDays, DroppedSuggestionsJson
<code>ItineraryDay</code>	ItineraryId, DayNumber, Date, Title, WeatherNote
<code>ItineraryItem</code>	DayId, PlaceName, Category, Description, DurationMinutes, EstimatedCostPerPerson, BookingSearchQuery, BookingType, IsCompleted

8B — Itinerary Request Statuses

"Open" → "Generating" → "Generated"

8C — Collaborative Flow (with suggestions)

Endpoint	Method	Description
----------	--------	-------------

POST /api/itinerary/request	Admin	Create itinerary request (opens suggestion phase)
POST /api/itinerary/{requestId}/suggest	Members	Add place suggestion
DELETE /api/itinerary/suggestion/{id}	Owner/Admin	Delete suggestion
POST /api/itinerary/suggestion/{id}/vote	Members	Toggle vote on suggestion
PATCH /api/itinerary/suggestion/{id}/review	Admin	Approve/reject suggestion
GET /api/itinerary/{requestId}/suggestions	Members	List all suggestions with votes
POST /api/itinerary/{requestId}/generate	Admin	Generate itinerary via Gemini AI
POST /api/itinerary/{requestId}/regenerate	Admin	Delete generated + regenerate
GET /api/itinerary/{requestId}/result	Members	Get full generated itinerary
PATCH /api/itinerary/item/{itemId}/complete	Members	Toggle item completed
DELETE /api/itinerary/request/{requestId}	Admin	Delete request + all related data

8D — Auto-Generate Flow (no suggestions)

Endpoint	Method	Description
POST /api/itinerary/request/auto-generate	Admin	Create request + generate instantly

- Accepts `AutoGenerateItineraryDto` (same fields as `CreateRequest` + optional `Note`)

- Skips suggestion/voting phase entirely
 - Returns `ItineraryResultDto` directly
-

8E — Core Generation Logic (`RunGenerationAsync`)

Shared private method used by both flows:

1. **Weather fetch** — Open-Meteo geocoding → forecast per trip day
2. **Suggestions query** — loads approved/neutral suggestions (empty for auto-generate → AI picks freely)
3. **Budget computation:**
 - `totalDays = EndDate - StartDate + 1`
 - `numberOfNights = totalDays - 1`
 - `tripVehicleCost = (dailyRental + dailyFuel) × vehicleCount × totalDays`
 - `tripHotelCost = (DailyHotelCostPerRoom × NumberOfRooms) × numberOfNights`
 - `remainingBudget = TotalBudget - tripVehicleCost - tripHotelCost`
4. **Prompt construction** — vehicle context + accommodation context + suggestions + weather + budget
5. **Gemini AI call** — structured JSON response
6. **JSON parsing** — cleans markdown fences, parses days/items
7. **DB persistence** — saves `GeneratedItinerary` → `ItineraryDay[]` → `ItineraryItem[]`
8. Sets `Status = "Generated"`

Error handling: Throws `InvalidOperationException` on Gemini failure → callers catch and reset status to `"Open"`

8F — Budget Context in AI Prompt

Scenario	AI Instruction
Own/rental vehicle	"Vehicle cost pre-budgeted — do NOT add transport items"

Public transport	"Include bus/auto/taxi as separate items with realistic cost"
Hotel provided	"Hotel pre-budgeted ($\text{₹X/night} \times \text{N rooms} \times \text{N nights}$) — do NOT add hotel items"
Free accommodation	"Free accommodation (friends/family) — do NOT add hotel items"

8G — Vehicle Auto-Calculation

If `VehicleCount` not provided, auto-calculated from `VehicleType` + group size:

Vehicle	Capacity
Scooty / Bike	2 per vehicle
Car	4 per vehicle
SUV	7 per vehicle
Traveller	1 vehicle total

8H — ItineraryResultDto Fields

```
TotalBudget, TotalDays, GroupSize
VehicleTotalCost, DailyVehicleCost, VehicleType, VehicleCount
HotelTotalCost, DailyHotelCost, NumberOfNights
EstimatedActivityCost, EstimatedTotalSpend, BudgetRemaining,
EstimatedCostPerPerson
Days[] → Items[] (with booking info, cost, category, completion status)
DroppedSuggestions[]
```

Bugs Fixed (this session)

Bug	Fix
-----	-----

<code>totalDays</code> always = 1 (dates swapped at data entry)	Validated <code>StartDate < EndDate</code> guidance
<code>totalDays</code> calc: <code>StartDate - EndDate</code> (negative)	Changed to <code>EndDate - StartDate</code>
<code>dailyVehicleCost = rental × fuel</code> (should be +)	Fixed to <code>rental + fuel</code>
<code>dailyHotelCost</code> ignored <code>NumberOfRooms</code> (line 214)	Changed to <code>DailyHotelCostPerRoom × NumberOfRooms</code>
Missing <code>await</code> on final <code>SaveChangesAsync</code> in <code>RunGenerationAsync</code>	Added <code>await</code>
<code>dto.Note / requestId</code> used in scope of <code>RunGenerationAsync</code> (wrong scope)	Changed to <code>note</code> param / <code>request.Id</code>
EF identity map cache: auto-generate returned <code>Days.Count = 0</code>	Added <code>_db.ChangeTracker.Clear()</code> before <code>GetItineraryResultAsync</code>
Missing <code>Status = "Generating"</code> in <code>AutoGenerateItineraryAsync</code>	Added to request initializer
Missing DB migration for <code>DailyHotelCostPerRoom</code> , <code>NumberOfRooms</code> columns	Ran <code>Add-Migration + Update-Database</code>

❑ Pending / Deferred Work

Itinerary Module

- ❑ **RegenerateAutoAsync** — Regenerate for auto-generate flow
 - Accepts `requestId` + `AutoGenerateItineraryDto` (allows updating trip details)
 - Deletes existing `GeneratedItinerary`, updates request fields, re-runs `RunGenerationAsync`
 - Route: `POST /api/itinerary/request/{requestId}/regenerate-auto`

☐ **Guard in `AutoGenerateItineraryAsync`** — Block if group already has ANY itinerary request (not just non-generated ones), with message "Use regenerate-auto instead"

General

- ☐ Angular frontend (not started)
 - ☐ Push notifications
 - ☐ Booking affiliate link integration
-

Key Design Patterns Used

- **`ServiceResult<T>`** — consistent success/failure wrapper across all services
- **Admin-only guards** — itinerary create/generate/delete, group management
- **Cascade delete** — manual ordering (votes → suggestions → items → days → itinerary → request)
- **Single `RunGenerationAsync` private method** — shared between collaborative and auto-generate, zero duplication
- **`ChangeTracker.Clear()`** — used after auto-generate to prevent EF identity map returning stale in-memory entities